# ADF Copy Activity Performance Tuning Runbook

## Purpose

This is the detailed companion to Quick_ADF_Copy_Activity_Performance_Tuning_Runbook. Use the Quick runbook when you need the smallest practical sequence of large-load tests. Use this full runbook when you need the reasoning behind a test, the SQL Managed Instance (SQL MI) signals to inspect, and the remediation path when a setting does not help.

> *Actively tuning a large load? Start with* Quick ADF Copy Activity Performance Tuning Runbook (Word) | Quick ADF Copy Activity Performance Tuning Runbook (PDF)*. Use* Quick Reference: Diagnosis-Driven Runbook (Word) | Quick Reference: Diagnosis-Driven Runbook (PDF) *when you need a symptom-to-next-action lookup.*

The working production context used while designing this runbook is a delimited-file load of approximately 134.04 GiB and 78,949,101 rows into SQL MI. The learning environment can use a much smaller representative file. The goal is to learn and prove the tuning method, not reproduce production scale in the free lab.

## 1. Operating principles

### 1.1 Change one material variable at a time

DIUs, write batch size, parallel copies, number of source files, number of simultaneous writers, table design, and MI capacity are different variables. If two are changed together, an improvement cannot be attributed reliably.

### 1.2 Diagnose before testing

Do not run a complete matrix of DIUs, batches, parallel-copy values, and file counts. Start with a baseline. Let the baseline evidence determine the next test.

### 1.3 Use a 10% useful-improvement threshold during exploration

For long-running or expensive loads, an exploratory change should normally improve end-to-end duration or throughput by at least 10% to justify another test in the same direction.

- >=10% improvement: retain the setting as a candidate; test one more level only if additional improvement matters.
- <10% improvement: stop that tuning branch unless cost or another operational requirement provides a separate reason to continue.
- >15% baseline drift: if the repeated baseline changes by more than 15%, treat the comparison as environmentally unstable and repeat only the baseline/candidate comparison.

### 1.4 Correctness is a gate

A faster load fails if the data is wrong. At minimum validate:

- expected versus actual row count;
- rejected/error rows;
- important key ranges or aggregates;
- duplicate behavior where business keys matter;
- file-partition row totals when a source file is split.

### 1.5 Measure ADF and SQL MI together

A setting is not successful merely because ADF pushes data faster. It can move the bottleneck into SQL MI transaction logging, locking, or storage. Capture enough SQL MI evidence to determine whether the sink remained healthy.

## 2. Baseline: the one run that chooses the next test

Run the current/default configuration once using a representative workload.

Record:

| Area | Measure | Why it matters |
|---|---|---|
| ADF | Copy duration and throughput | Primary end-to-end performance result |
| ADF | Rows/data copied and rejected | Correctness and workload-size confirmation |
| ADF | DIUs actually used | Shows the data-movement compute ADF used |
| ADF | Parallel copies / source-sink connections | Shows actual concurrency |
| SQL MI | WRITELOG | Indicates waiting for transaction-log flushes |
| SQL MI | INSTANCE_LOG_RATE_GOVERNOR | Indicates SQL MI log-write rate governance is delaying work |
| SQL MI | Active log usage and log reuse reason | Shows whether transactions or backup/reuse behavior retain log space |
| SQL MI | Log write latency / observed log MiB/s | Shows pressure on the transaction-log write path |
| SQL MI | Data-file IOPS/latency | Shows whether data storage is becoming the constraint |
| Cost | Cost/load and cost/GiB when available | Prevents paying materially more for negligible speed |

Use the SQL monitoring scripts in sql/monitoring/ as supporting evidence. Script 07 measures wait deltas for the load window, and script 08 captures live file IOPS, throughput, and latency.

### Baseline routing table

| What the baseline shows | Likely issue | Smallest next test |
|---|---|---|
| Low ADF throughput; SQL MI waits and I/O are low | ADF/source path may have unused capacity | One higher-DIU test |
| High WRITELOG; low INSTANCE_LOG_RATE_GOVERNOR | Commit/log-flush behavior may be limiting | One larger-batch test |
| Sustained INSTANCE_LOG_RATE_GOVERNOR | SQL MI log-write capacity is limiting | Do not lead with more DIUs/writers; simplify sink or test MI capacity |
| One large source file; SQL MI has headroom | Source serialization may be limiting | One 4-file comparison |
| Multiple files; low ADF concurrency; SQL MI has headroom | Copy parallelism may be limiting | One parallel-copies comparison |
| Lock waits rise with multiple writers | Sink writers are contending | Reduce concurrency |
| Data-file latency/IOPS rises while throughput plateaus | SQL MI storage/data I/O is limiting | Investigate MI storage/capacity |

# 3. DIU tuning

## What DIUs are testing

Data Integration Units allocate data-movement resources to Copy Activity on Azure Integration Runtime. The purpose of the test is not to find the largest DIU value. It is to determine whether additional ADF data-movement resources still improve useful throughput.

## Procedure

1. Keep batch size, file layout, target table, and parallel-copy setting unchanged.
2. Record the baseline DIU setting and DIUs actually used.
3. Run one meaningful higher DIU value.
4. Compare throughput, duration, WRITELOG, INSTANCE_LOG_RATE_GOVERNOR, and log/data I/O latency.
5. Test one additional DIU level only if the first increase improved performance by at least 10% and the remaining runtime warrants more tuning.

## Interpret the result

| Observed result | What it means | Action |
|---|---|---|
| Throughput improves >=10%; SQL MI signals remain comparable | ADF compute was limiting or contributing | Keep candidate; optionally test one more DIU level |
| Throughput improves <10% | Added ADF compute is not producing enough useful work | Stop DIU testing |
| WRITELOG or log latency climbs while throughput changes little | SQL MI log flush/write path is absorbing added pressure | Stop increasing DIUs; investigate log/batch path |
| INSTANCE_LOG_RATE_GOVERNOR rises sharply while throughput improves <10% | SQL MI is increasingly delayed by its transaction-log write-rate limit | Stop increasing DIUs/writers; move to SQL MI/log-capacity path |
| Data-file latency rises while throughput plateaus | SQL MI storage is becoming limiting | Stop DIU escalation; investigate MI I/O/capacity |
| Cost/GiB rises materially for <10% gain | Additional DIUs are economically inefficient | Prefer lower setting |

Example:

8 DIUs: 100 MiB/s, 1 second INSTANCE_LOG_RATE_GOVERNOR
16 DIUs: 104 MiB/s, 75 seconds INSTANCE_LOG_RATE_GOVERNOR

The 8-to-16 change produced only 4% more throughput while SQL MI spent substantially more time under log-rate governance. Keep 8 DIUs as the candidate and stop increasing ADF write pressure.

# 4. Working with SQL MI FULL recovery

SQL MI remains in FULL recovery. This runbook does not depend on switching to SIMPLE or BULK_LOGGED recovery. Tune the load around FULL recovery by controlling transaction size, commit frequency, target-table overhead, and concurrent write pressure.

## What to monitor

Before and during the load, look at:

- active transaction-log usage;
- log_reuse_wait_desc;
- WRITELOG;
- INSTANCE_LOG_RATE_GOVERNOR;
- log write latency and observed log MiB/s;
- long-running active transactions;
- rollback duration after a failed load.

### What specific conditions mean

- ACTIVE_TRANSACTION persists and active log grows after increasing batch size: a larger transaction is retaining log space longer. Reduce batch size unless the performance benefit clearly justifies the recovery risk.
- WRITELOG rises sharply without a proportional throughput gain: sessions are spending more time waiting for log flushes. Test batch/commit behavior rather than more DIUs.
- INSTANCE_LOG_RATE_GOVERNOR rises sharply while throughput plateaus: the workload is reaching SQL MI log-write capacity. More DIUs or writers are not the next fix.
- The log grows during only one benchmark because it was not pre-sized: the run is measuring autogrowth as well as ADF performance. Stabilize file allocation before comparing results.

### When log tuning is exhausted

If controlled batch sizing and writer concurrency do not remove the log bottleneck:

6. Compare the current target with the simplest practical empty staging heap: no unnecessary indexes, foreign keys, or triggers.
7. Measure total workflow time, including any later insert/merge/index work, not only the ADF copy step.
8. If the simple heap still shows substantial INSTANCE_LOG_RATE_GOVERNOR and more ADF resources add <10% throughput, stop tuning ADF.
9. The next test is SQL MI capacity/configuration. Temporarily scale the relevant MI capacity and rerun the same workload with the same ADF settings once.
10. If throughput improves >=10% and log-governor waiting drops materially, MI capacity was constraining the load. Evaluate the cost and whether temporary scale-up is practical.
11. If MI scaling produces little gain, investigate the next measured constraint: storage/data I/O, network path, locking, or target-table work. Do not return to blind DIU testing.

## 5. Write batch sizing

### What batch sizing is testing

Batch size changes commit frequency and transaction size. Under FULL recovery, larger batches can reduce commit overhead but can also retain more active log, lengthen rollback, and increase recovery exposure.

### Procedure

12. Keep DIU, source layout, parallelism, and target unchanged.
13. Baseline the current/default batch.
14. If WRITELOG/commit behavior appears important and log-rate governance is not already dominant, run one larger batch. For an unset/small baseline, 50,000 rows is a practical first comparison.
15. Compare duration/throughput, WRITELOG, active log usage, log latency, and rollback behavior.
16. Test one larger value only if the first batch change improves performance >=10% and remains operationally acceptable.

### Interpret the result

| Result | Meaning | Action |
|---|---|---|
| >=10% faster; log usage/latency remain comparable | Commit overhead was contributing | Keep candidate; optionally test one larger batch |
| <10% faster | Transaction-size change did not create enough value | Stop; keep smaller/default batch |
| Active log rises sharply; ACTIVE_TRANSACTION persists | Larger transaction retains log longer | Reduce batch size |
| Rollback becomes substantially longer | Larger transaction increased recovery exposure | Treat as production cost; reduce unless gain justifies it |
| INSTANCE_LOG_RATE_GOVER | MI log-rate capacity, not commit | Stop batch experimentation; |

| | | |
|---|---|---|
| NOR becomes dominant | frequency, is limiting | move to MI/log-capacity path |

## 6. Single file versus multiple files

### What file partitioning is testing

The objective is to determine whether source-file serialization is preventing ADF from using available source-side parallelism. It is not an instruction to create many simultaneous SQL writers.

### Procedure

17. Keep the canonical one-file dataset unchanged.
18. Use split_delimited_file.py to create 4 near-equal logical-record files.
19. Confirm the sum of the partition row counts equals the original.
20. Use one Copy Activity against the multi-file source.
21. Keep DIUs, batch size, and target design unchanged.
22. Compare end-to-end duration/throughput and SQL MI pressure.

### Interpret the result

- 4 files are >=10% faster and SQL MI remains comparable: file-level/source parallelism helped. Stop at 4 unless the remaining runtime justifies one 8-file test.
- Improvement is <10%: do not automatically test 8 and 16 files. Partitioning is not a first-order fix.
- ADF source activity rises while end-to-end throughput plateaus and SQL MI waits increase: source parallelism has saturated the sink. Stop adding files/writers.
- 8 files are slower than 4: partition overhead or downstream contention exceeds the parallelism benefit. Use 4 for that tested configuration.

For production, do not repartition an external feed solely because the lab improved. Consider upstream ownership, naming/arrival contracts, restart behavior, storage transactions, and partial-file failure handling.

## 7. Parallel copies and writer concurrency

Parallel copies and simultaneous database writers must be distinguished.

### Procedure

23. Use this test only when multiple source files/partitions exist and SQL MI has headroom.
24. Baseline current/Auto parallel-copy behavior.
25. If effectively serial, test 2 parallel copies. Otherwise test the next practical value.
26. Test 4 only if 2 improves end-to-end throughput >=10% and SQL MI remains healthy.

### Stop conditions

- Next concurrency level improves <10%.
- LCK_M_X or LCK_M_IX rises sharply while throughput plateaus: writers are contending for locks; reduce concurrency.
- INSTANCE_LOG_RATE_GOVERNOR rises sharply while throughput changes little: concurrent writers are pushing SQL MI into log-rate governance; reduce concurrency or test MI capacity.
- Source throughput grows without end-to-end improvement: the sink or another downstream component is limiting.

## 8. Target table design and staging

Do not assume staging is always faster, and do not assume it is needed only for transformation.

Use a staging-heap comparison when the final target has meaningful insert overhead from indexes, constraints, triggers, or concurrent access.

Compare:

```
Direct:
Storage -> ADF -> Final table

Staged:
Storage -> ADF -> Empty heap -> set-based SQL work -> Final table
```

Measure the complete staged workflow:

- ADF heap load;
- set-based insert/merge;
- index creation/maintenance;
- cleanup;
- total elapsed time;
- transaction-log behavior.

A staging design passes only when the total workflow provides a useful operational improvement, not merely when the first ADF step is faster.

## 9. SQL MI data-file I/O and instance capacity

Use scripts 06 and 08 plus wait information to distinguish transaction-log pressure from data-file/storage pressure.

### Evidence of data-file/storage pressure

- live data-file IOPS rises toward a verified platform/configuration limit;
- read/write latency rises across repeated active-load samples;
- PAGEIOLATCH_*/I/O waits increase;
- ADF pressure increases while end-to-end throughput plateaus.

Do not resize files or service tiers solely from one five-second latency sample. Confirm the condition with repeated samples and ADF end-to-end throughput.

If storage/MI capacity is the measured constraint, the next test is a controlled MI configuration/capacity change with the same ADF workload, not another random ADF setting change.

## 10. Confirmation and production acceptance

### Lab confirmation

Do not repeat every exploratory configuration three times. Once a likely winner exists:

27. Rerun the original baseline once.
28. Rerun the candidate once.
29. Accept the lab candidate if it still improves >=10%, data is correct, and SQL MI/cost behavior is acceptable.
30. If the repeated baseline drifted >15%, repeat only baseline versus candidate under stable conditions.

### Production acceptance

Lab results create a hypothesis. Production proves it.

31. Capture the current production execution as P0 using the same core metrics.
32. Select the single change supported by the diagnosis.
33. Apply that one change in the approved window.
34. Validate correctness before performance.
35. Compare duration, throughput, SQL MI waits/latency, reliability, and cost with P0.

Classify the result:

| Outcome | Meaning |
|---|---|
| PASS | >=10% useful improvement, correct data, no unacceptable SQL MI/cost/reliability regression |
| FAIL | No useful improvement, incorrect data, or operational regression; rollback |
| INVESTIGATE | Faster, but a new bottleneck, cost increase, or risk requires explanation before promotion |

Only a PASS becomes the new production baseline.

## 11. Benchmark record

For a meaningful test, retain:

- Test ID / environment / timestamp
- Source GiB and row count
- Source file count
- Configured and actual DIUs
- Parallel copies / connections
- Write batch size
- Target table design
- Copy duration and throughput
- Rows copied/rejected
- Pre/during/post log usage
- WRITELOG wait delta
- INSTANCE_LOG_RATE_GOVERNOR wait delta
- Observed log write MiB/s and write latency
- Relevant data-file IOPS/latency
- Lock waits when concurrency is tested
- Cost per load / cost per GiB when available
- Correctness result
- PASS / FAIL / INVESTIGATE
- Reason for the next action

The final field matters: document why the evidence caused the next test. That prevents tuning from becoming a sequence of unexplained setting changes.

## 12. Writing and decision standard

Every tuning instruction should state:

36. What was measured?
37. What obvious technical issue does it indicate?
38. What is the smallest next test or action?

Avoid vague wording such as "becomes material," "looks high," or "appears constrained" without defining the measured comparison that triggered the decision.